

TICKET BOOKING SYSTEM DESIGN x {MOVIE}

Book My Show

DISTRICT

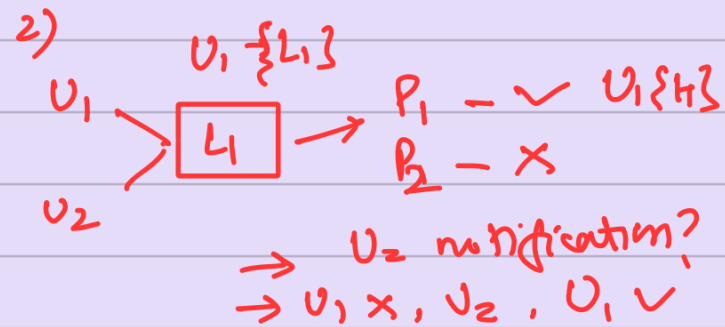
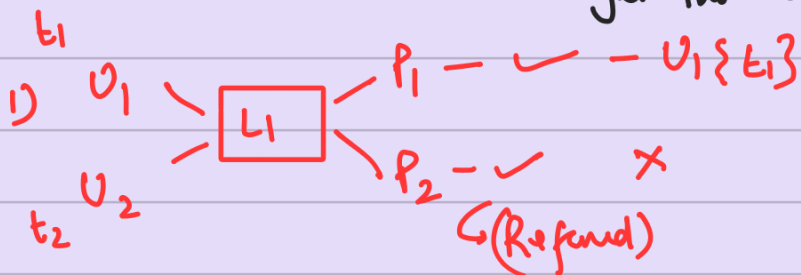
Requirements

- (*) 1) Book the ticket
- 2) Search movie, view Details
- 3) Search All the cinema (PVR) near you
- * 4) Highly Concurrent
- (*) 5) Payment (not going in Details)
- (*) 6) Comment (not going in Details)
- (*) 7) Send ticket (notification → (not going in Detail))

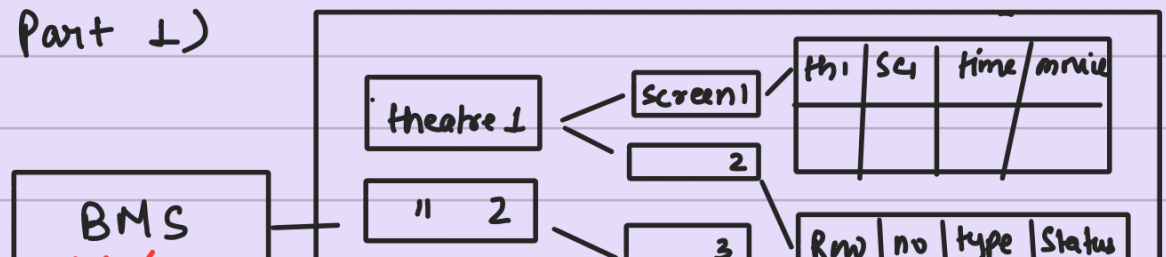
Approach :-

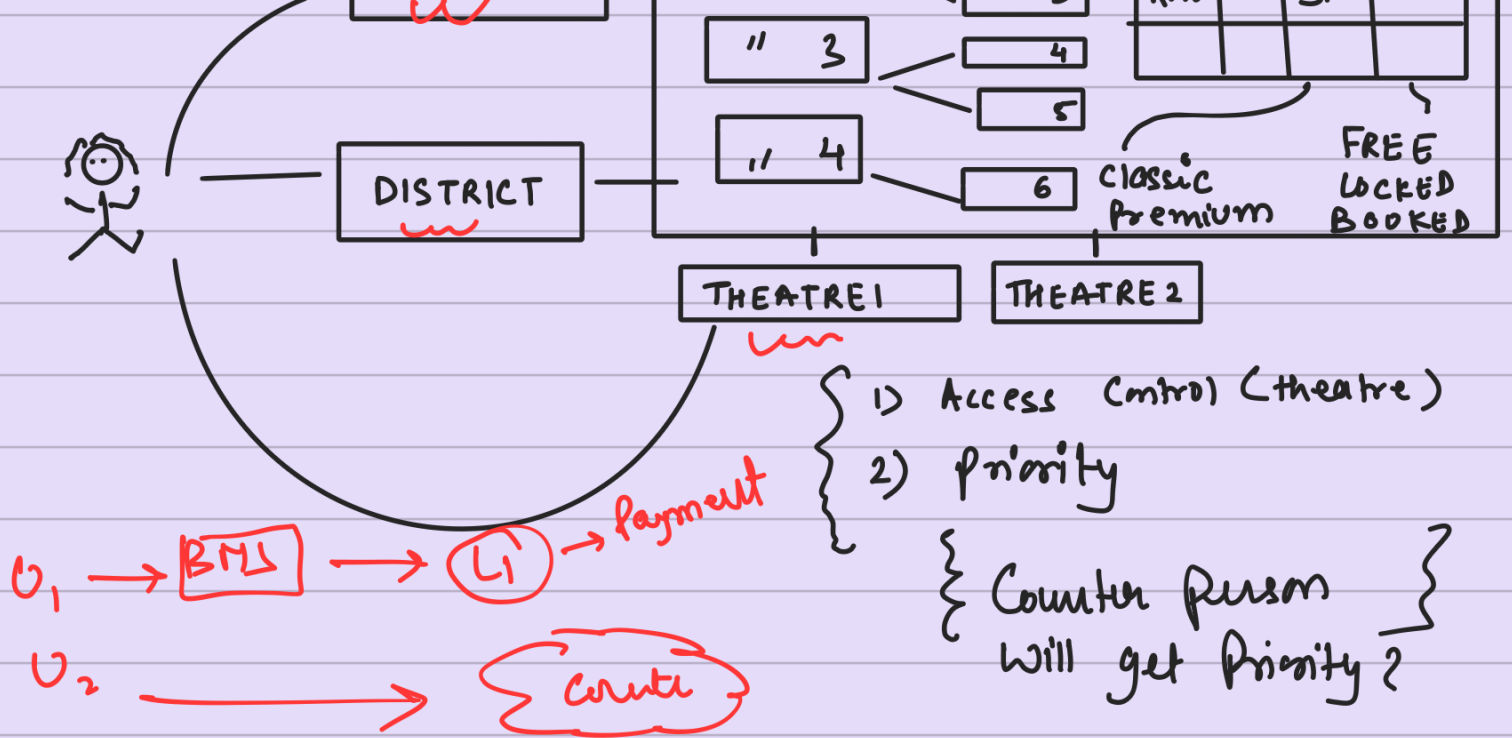
1) ticket claim after Payment confirmed.

d) ticket claim with some locking strategy.
(*) the person who come first will get the lock.



Concept 1 (Book the ticket Part 1)





Concept 2 (Understanding DB Relation)

- 1) All the above table { Relational DB }
- 2) movie → { mongo DB }
- Document type Data. → Raw info

{ name, -
actors - [,]
co-actor list, [, , ,]
ratings: [imdb: -
BMS: -]

active -

Mapping table

3)

theatreid	screenid	time	movie

Concept 3 (Search movie all the cinema, near you)

1)



2) Ask where do you want to Book ticket.

1) → search on the basis of city

2) → Search on the basis of Coordinate . distance .

3) → By name Search

{ fuzzy Search }
{ soundex Search }

Concept 4 :- (Highly Concurrency)

{ Concurrency Control }

Isolation level	Dirty Read	Non-Repeatable Read	Phantom Read	Control
Read uncommitt	Yes	Yes	Yes	Optimistic ↑
Read commit	No	Yes	Yes	Optimistic ↓
Repeatable Read	No	No	Yes	Pessimistic ↓
Serializable	No	No	No	Pessimistic ↑

1) which one do you need?

1) Pessimistic

pro

1) prevent double booking

2) lock the seat During booking

{ low level locking }

Cons

1) Deadlock

2) Slow

Concept 5 :- { UI }

tnid	scnum	Seatid	Status	Ucnid	Starttime	expiry
✓	✓	123	Free	null	null	null
			locked <u>mm</u>	123	x	(x+10)

(Status = free x)

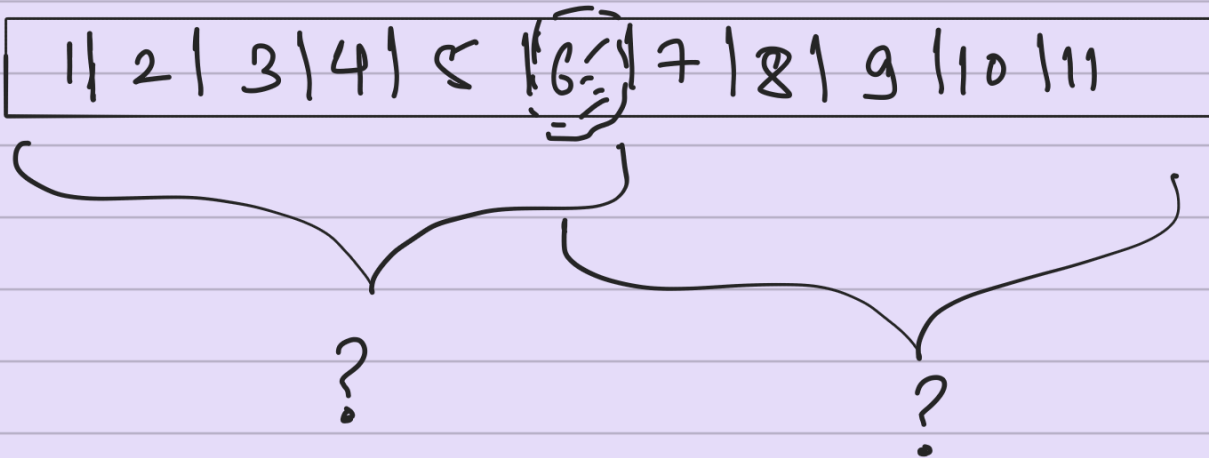
Status != Booked and

U1 - locked - (x - x+10) (x+12)

(user) time > expirytime { Free mm }

Stored proc, Redis, DB trigger
Cron.

Concepts - 6



→ I had to rollback.

→ Should we give them middle ground seats of booking available

final Design

